🚀 Collections Framework Masterclass

Chapter 22 — HashMap Internals, Collisions, Rehashing & Treeification

> **Interview Importance:** ⭐⭐⭐⭐⭐

This is one of the **highest-value Java Collections chapters for interviews**.

If an interviewer asks:

> **"Explain how HashMap works internally."**

you should be able to walk through this entire chain:

```
put(key, value)
      ↓
hashCode()
      ↓
hash spreading
      ↓
bucket index
      ↓
empty bucket?
   ↙          ↘
 YES           NO
  ↓             ↓
insert       collision
                ↓
          equals() comparison
             ↓
       ┌─────┴──────┐
       ↓            ↓
    same key      different key
       ↓            ↓
    replace       add node
                    ↓
              too many collisions?
                    ↓
               treeification
```

Then understand:

```
size
 ↓
threshold exceeded
 ↓
resize
 ↓
capacity doubles
 ↓
entries redistributed
```

Let's break this down deeply without repeating concepts already covered in previous chapters.

---

1. What Is HashMap?

`HashMap<K,V>` is a hash-table-based implementation of `Map`.

```
Map<String, Integer> map =
    new HashMap<>();
```

It provides average:

```
get()     → O(1)
put()     → O(1)
remove()  → O(1)
```

assuming good hash distribution.

The important word is:

> **Average**

Poor hashing or extreme collisions can change the performance characteristics.

---

2. HashMap Internal Structure

Conceptually, modern Java HashMap looks like:

```
HashMap
   |
   ↓
Node<K,V>[] table
   |
   ├── bucket 0 → null
   ├── bucket 1 → Node
   ├── bucket 2 → null
   ├── bucket 3 → Node → Node
   ├── bucket 4 → null
   └── ...
```

Each bucket can contain:

```
nothing
```

or:

```
linked nodes
```

or, when collision conditions warrant:

```
tree bin
```

---

3. Node Structure

Conceptually, a HashMap node contains:

```java
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;
}
```

So each entry essentially stores:

```
hash
key
value
next
```

The actual JDK implementation contains additional details, but this is the important interview model.

---

4. What Happens During `put()`?

Suppose:

```
map.put("Java", 100);
```

The broad process is:

```
"Java"
   ↓
hashCode()
   ↓
hash spreading
   ↓
bucket index
   ↓
inspect bucket
   ↓
insert/update
```

The two methods/concepts interviewers want you to understand are:

```
hashCode()
equals()
```

---

5. Step 1 — `hashCode()`

For the key:

```
"Java"
```

Java first obtains:

```
key.hashCode()
```

This produces an integer.

For a custom object:

```
class Employee {
    int id;

    @Override
    public int hashCode() {
        return id;
    }
}
```

HashMap uses that hash information to determine where the entry should go.

---

6. `hashCode()` Does NOT Directly Give the Bucket

A common interview mistake is saying:

> "HashMap uses hashCode as the bucket index."

Not exactly.

`hashCode()` returns an integer that can be any `int` value.

HashMap needs to transform that value into a valid table index.

Modern HashMap also performs a hash-spreading step.

---

7. Hash Spreading

Conceptually, Java 8+ HashMap uses:

```
h ^ (h >>> 16)
```

where:

```
h = key.hashCode()
```

So:

```
original hash
     ↓
upper bits mixed into lower bits
     ↓
spread hash
```

Why?

Because bucket selection relies heavily on lower bits when the table size is a power of two.

Mixing higher bits into lower bits helps distribute keys more effectively.

---

8. Why Mix the Upper Bits?

Suppose two keys have hash codes where:

```
lower bits are identical
```

but:

```
upper bits differ
```

If only the lower bits influenced bucket selection, those differences could be wasted.

Hash spreading:

```
h ^ (h >>> 16)
```

mixes information from the high bits into the low bits.

This improves distribution.

---

9. Bucket Index Calculation

This is one of the most frequently asked HashMap questions.

For a table length:

```
n
```

the bucket index is conceptually:

```
(n - 1) & hash
```

So:

```
int index = (n - 1) & hash;
```

---

10. Why Not `%`?

You might expect:

```
hash % n
```

HashMap instead uses:

```
(hash & (n - 1))
```

when `n` is a power of two.

For example:

```
n = 16
n - 1 = 15
```

Binary:

```
16 → 10000
15 → 01111
```

Therefore:

```
hash & 01111
```

extracts the relevant lower bits.

Bitwise masking is efficient and works cleanly because HashMap maintains power-of-two capacities.

---

11. Why Must Capacity Be a Power of Two?

This is a classic interview question.

If:

```
capacity = 16
```

then:

```
capacity - 1 = 15
```

Binary:

```
00001111
```

Masking becomes straightforward:

```
hash & 00001111
```

This efficiently maps hashes to indices:

```
0 → 15
```

A non-power-of-two capacity would not give the same useful bitmask behavior.

---

12. Complete Hash Calculation

Think:

```
key
 ↓
key.hashCode()
 ↓
h ^ (h >>> 16)
 ↓
(n - 1) & hash
 ↓
bucket index
```

This sequence is extremely important.

---

13. Example

Suppose:

```
hash = 101101011010
```

and:

```
capacity = 16
```

Then:

```
n - 1 = 15
```

or:

```
1111
```

The final bucket is determined from the relevant lower bits through:

```
hash & 1111
```

The exact numeric result isn't important.

The mechanism is.

---

14. What If the Bucket Is Empty?

Suppose:

```
index = 5
```

and:

```
table[5] == null
```

Then HashMap can simply insert:

```
table[5] → Node
```

Conceptually:

```
bucket 5
   ↓
Node("Java", 100)
```

Done.

---

15. What If the Bucket Is Not Empty?

Then we have a:

Hash collision

Example:

```
Key A → bucket 5
Key B → bucket 5
```

Even if:

```
A != B
```

they can still map to the same bucket.

This is completely normal.

---

16. Hash Collision

A collision means:

```
different keys
      ↓
same bucket
```

It does **not** necessarily mean:

```
hashCode(A) == hashCode(B)
```

Two different hash codes can still map to the same bucket because the final bucket index uses only part of the hash.

Therefore:

> Same bucket does not imply same hash code.

---

17. Collision Example

Suppose:

```
Key A → hash 10
Key B → hash 26
```

With a small table, both could map to the same bucket.

Conceptually:

```
bucket 2
   ↓
A → B
```

HashMap stores both.

---

18. How Does HashMap Distinguish Colliding Keys?

It uses:

```
hash
+
equals()
```

Conceptually:

```
same bucket
    ↓
compare stored hash
    ↓
if necessary compare keys with equals()
```

The `equals()` method determines whether the key is actually the same logical key.

---

19. Collision ≠ Duplicate Key

This distinction is critical.

Collision

```
different keys
→ same bucket
```

Duplicate key

```
same logical key
→ same bucket
→ equals() == true
```

A collision can contain multiple entries.

A duplicate key replaces the existing value.

---

20. Example

```
map.put("A", 10);
map.put("B", 20);
```

If A and B collide:

```
bucket
 ↓
A → B
```

Both remain.

But if:

```
map.put("A", 30);
```

then:

```
A → 30
B → 20
```

The existing A mapping is updated.

---

21. Why `equals()` and `hashCode()` Must Agree

The contract is:

If `a.equals(b)` is true, `a.hashCode()` must equal `b.hashCode()`.

Otherwise HashMap can place logically equal keys into different buckets.

---

22. Example of Broken Contract

Suppose:

```
class Employee {
    int id;

    @Override
    public boolean equals(Object o) {
        Employee e = (Employee) o;
        return id == e.id;
    }

    @Override
    public int hashCode() {
        return System.identityHashCode(this);
    }
}
```

Now:

```
e1.equals(e2) → true
```

but:

```
e1.hashCode() != e2.hashCode()
```

can occur.

HashMap's assumptions are broken.

---

23. Correct Contract

If:

```
a.equals(b) == true
```

then:

```
a.hashCode() == b.hashCode()
```

must also be true.

But the reverse is NOT required.

This is an extremely common interview question.

---

24. Same Hash ≠ Equal Objects

Two objects can have:

```
same hashCode()
```

but:

```
equals() == false
```

That's a collision.

Example:

```
A.hashCode() → 100
B.hashCode() → 100

A.equals(B) → false
```

HashMap stores both.

---

25. `equals()` and `hashCode()` Decision Flow

When looking for a key:

```
calculate hash
      ↓
find bucket
      ↓
compare candidate hash
      ↓
compare keys
      ↓
equals()
      ↓
same key?
```

Conceptually:

```
hashCode() → locate area
equals()   → identify exact key
```

This is one of the best ways to remember their roles.

---

26. `get()` Internals

Suppose:

```
map.get(key);
```

Conceptually:

```
key
 ↓
hashCode()
 ↓
spread hash
 ↓
bucket index
 ↓
bucket
 ↓
compare candidate
 ↓
equals()
```

```
 ↓
return value
```

If no matching key exists:

```
null
```

---

27. `remove()` Internals

Similarly:

```
map.remove(key);
```

does:

```
hash
 ↓
bucket
 ↓
search
 ↓
equals()
 ↓
remove matching node
```

If the key doesn't exist:

```
no mapping removed
```

---

28. Java 7 vs Java 8 HashMap

This is a very important interview topic.

Java 7

Collision chains were linked lists.

```
bucket
 ↓
Node → Node → Node
```

Java 8+

Heavy collision chains can become:

```
TreeBin
 ↓
Red-Black Tree
```

This is one of the major improvements.

---

29. Why Treeification Was Introduced

Imagine:

```
bucket
 ↓
A → B → C → D → E → F → G → H → ...
```

Searching could become:

```
O(n)
```

for a heavily collided bucket.

Treeification changes the structure to a balanced tree.

Conceptually:

```
      D
     /
    B   F
   / \ /
  A  C E  G
```

Lookup can then become approximately:

```
O(log n)
```

within that tree.

---

30. Red-Black Tree

The tree structure used for treeified HashMap bins is a:

> **Red-Black Tree**

It is self-balancing.

The goal is to prevent the collision structure from degenerating into a long linear chain.

---

31. Treeification Threshold

A key implementation constant is:

```
TREEIFY_THRESHOLD = 8
```

This means treeification is considered when a bin becomes sufficiently populated.

But there is an important condition.

HashMap doesn't immediately treeify every bin that reaches eight nodes.

---

32. Minimum Treeify Capacity

Another important constant:

```
MIN_TREEIFY_CAPACITY = 64
```

Conceptually:

```
bin gets crowded
       ↓
```

```
table capacity < 64?
       ↓
resize instead of treeify
```

Once the table is sufficiently large, treeification can occur when the collision threshold is reached.

---

33. Why Resize Instead of Treeify?

A small table naturally causes more collisions.

Suppose:

```
capacity = 16
```

and one bucket contains many entries.

Rather than immediately building a tree, HashMap may prefer:

```
resize
 ↓
capacity = 32
```

or:

```
capacity = 64
```

This spreads entries across more buckets.

So:

> **Treeification is not simply "8 nodes means tree."**

That is an incomplete interview answer.

---

34. Correct Interview Answer

If asked:

> "When does HashMap treeify?"

Say:

> When a bin becomes sufficiently large, around the treeification threshold of 8 nodes, HashMap may convert it to a tree, provided the table has reached the minimum capacity of 64. Otherwise, resizing is preferred.

That's much stronger.

---

35. Untreeification

A tree bin can later be converted back to a linked structure when the number of nodes becomes sufficiently small.

A commonly cited implementation threshold is:

```
UNTREEIFY_THRESHOLD = 6
```

The exact transition conditions are implementation details, but the interview concept is:

```
many collisions
 → tree

entries decrease
 → potentially linked list
```

---

36. Why Not Keep the Tree Forever?

A tree has more structural overhead than a simple linked node chain.

If the bin becomes small again:

```
tree
 ↓
too few entries
 ↓
linked representation may be more efficient
```

So HashMap can reduce unnecessary overhead.

---

37. Treeification Is Per Bucket

This is important.

HashMap doesn't become:

```
entire Map → Red-Black Tree
```

Instead:

```
bucket 1 → linked list
bucket 2 → empty
bucket 3 → tree
bucket 4 → linked list
```

Only collision-heavy bins are treeified.

---

38. HashMap Is Still a Hash Table

Even with tree bins:

```
HashMap
 ↓
array of buckets
 ↓
some buckets may contain trees
```

It doesn't become a TreeMap.

---

39. HashMap vs TreeMap

HashMap

```
hash table
 ↓
O(1) average lookup
```

TreeMap

```
Red-Black Tree
 ↓
O(log n)
```

HashMap's treeification is only a mechanism for handling collisions inside individual buckets.

---

40. Load Factor

Another extremely important concept:

> **Load factor controls when HashMap resizes.**

Default load factor:

```
0.75
```

Conceptually:

```
threshold = capacity × loadFactor
```

Example:

```
capacity = 16
load factor = 0.75

threshold = 16 × 0.75
          = 12
```

Once the size crosses the threshold, resizing occurs.

---

41. Why 0.75?

It's a compromise between:

```
memory usage
```

and:

```
collision frequency
```

Lower load factor:

```
more buckets
↓
fewer collisions
↓
more memory
```

Higher load factor:

```
fewer buckets
↓
more collisions
↓
less memory
```

The default `0.75` is a practical trade-off.

---

42. Capacity vs Size

These are different.

Capacity

Number of buckets:

```
16
```

Size

Number of mappings:

```
10
```

Example:

```
capacity = 16
size = 10
```

The Map has 16 buckets but only 10 entries.

---

43. Threshold

Threshold determines approximately when resizing is triggered:

```
threshold = capacity × load factor
```

For:

```
capacity = 16
load factor = 0.75
```

threshold:

```
12
```

---

44. Resize

When:

```
size > threshold
```

HashMap grows.

For the normal growth path:

```
16 → 32 → 64 → 128
```

Capacity generally doubles.

---

45. Why Double the Capacity?

Doubling allows HashMap to efficiently redistribute entries.

Because capacity remains a power of two, the new bucket position can be determined efficiently.

This is an important Java 8+ implementation optimization.

---

46. Rehashing

The term "rehashing" is often used loosely.

Strictly speaking, when HashMap resizes, it doesn't necessarily recompute each key's original `hashCode()` from scratch.

Instead, it redistributes entries based on the existing hash information and the new table capacity.

Therefore, a better modern-Java explanation is:

> **Resize + redistribution**, rather than imagining every key's hashCode being recalculated.

---

47. The Important Resize Trick

Suppose:

```
old capacity = 16
new capacity = 32
```

For an existing entry, its new index is related to:

```
old index
```

and:

```
old index + old capacity
```

depending on a particular hash bit.

Conceptually:

```
old bucket 5
     ↓
new bucket 5
or
new bucket 21
```

because:

```
21 = 5 + 16
```

This is one of the clever optimizations in Java 8 HashMap resizing.

---

48. Why Only One Bit Matters During Doubling?

Because:

```
new capacity = old capacity × 2
```

The mask gains one additional bit.

Therefore each existing entry can generally remain in:

```
same index
```

or move to:

```
old index + old capacity
```

depending on the relevant hash bit.

---

49. Example

Suppose:

```
old capacity = 16
```

An entry is currently at:

```
bucket 3
```

After resizing to:

```
32
```

it can go to:

```
3
```

or:

```
19
```

because:

```
3 + 16 = 19
```

This avoids recalculating the complete bucket placement from scratch.

50. Resize Visualization

Before:

```
Capacity = 16

bucket 3
  ↓
A → B

bucket 7
  ↓
C

bucket 12
  ↓
D → E
```

After:

```
Capacity = 32

bucket 3
  ↓
A

bucket 19
  ↓
B

bucket 7
  ↓
C

bucket 12
  ↓
D

bucket 28
  ↓
E
```

Exact distribution depends on hash bits.

---

51. Initial Capacity

When you write:

```
new HashMap<>(16);
```

you're specifying an initial-capacity-related sizing expectation, not necessarily saying:

> "Allocate exactly 16 buckets immediately."

Modern HashMap is lazily initialized.

The actual table allocation happens when needed, typically on the first insertion.

---

52. Lazy Initialization

This is an important implementation detail.

```
HashMap<String, Integer> map =
    new HashMap<>();
```

Creating the HashMap does not necessarily allocate the full bucket table immediately.

The table is initialized when the Map first needs storage.

---

53. Default Capacity

The commonly documented/default initial capacity is:

```
16
```

with:

```
load factor = 0.75
```

The table itself is lazily initialized.

54. Default Threshold

Initially, internal threshold handling is tied to lazy table allocation.

Once the table is initialized:

```
capacity = 16
load factor = 0.75
```

then the resize threshold becomes approximately:

```
12
```

Don't oversimplify the constructor state by saying the threshold is always 12 immediately after object creation.

---

55. Constructor Options

Common constructors:

```
new HashMap<>();
```

```
new HashMap<>(32);
```

```
new HashMap<>(32, 0.75f);
```

```
new HashMap<>(existingMap);
```

---

56. Choosing Initial Capacity

If you know approximately how many entries will be inserted, supplying a suitable initial capacity can reduce resizing.

For example:

```
Map<String, User> users =
    new HashMap<>(expectedCapacity);
```

But be careful:

> Constructor capacity is not necessarily equal to the eventual internal table length at construction time.

HashMap rounds sizing appropriately to its power-of-two strategy.

---

57. Load Factor Trade-Off

Lower load factor

Example:

```
0.50
```

Means:

```
more buckets
↓
fewer collisions
↓
more memory
```

Higher load factor

Example:

```
0.90
```

Means:

```
fewer buckets
↓
potentially more collisions
```

```
↓
less table memory
```

The default:

```
0.75
```

is a practical balance.

---

58. Does Higher Load Factor Always Mean Faster?

No.

Higher load factor can increase collisions.

That can lead to:

```
longer bins
more comparisons
potentially more tree bins
```

The correct load factor depends on workload and memory/performance trade-offs.

---

59. Does Lower Load Factor Always Mean Faster?

No.

You consume more memory and may get diminishing returns.

Again:

```
performance
+
memory
```

must be balanced.

---

60. `put()` — Complete Flow

Now combine everything.

Suppose:

```
map.put(key, value);
```

Flow:

```
1. Is table initialized?
       ↓
2. Calculate/spread hash
       ↓
3. Calculate bucket index
       ↓
4. Is bucket empty?
       ↓
   YES → insert
       |
      NO
       ↓
5. Compare existing key
       ↓
6. Same key?
   ↙          ↘
 YES          NO
  ↓            ↓
replace      collision
               ↓
       traverse linked/tree bin
               ↓
          insert new node
               ↓
      collision threshold?
               ↓
         treeify if eligible
               ↓
        size > threshold?
               ↓
             resize
```

This is the answer interviewers are looking for.

---

61. Duplicate Key During `put()`

Suppose:

```
map.put("A", 10);
map.put("A", 20);
```

HashMap finds:

```
same hash
+
keys equal
```

Therefore:

```
A → 20
```

Size remains:

```
1
```

---

62. New Key During Collision

Suppose:

```
A and B
 ↓
same bucket
```

but:

```
A.equals(B) == false
```

Then:

```
bucket
 ↓
A → B
```

Size increases.

---

63. `get()` Does Not Modify the Map

A normal:

```
map.get(key);
```

does not add an entry.

It simply:

```
hash
 ↓
bucket
 ↓
search
 ↓
return
```

---

64. `get()` and Null Values

HashMap permits null values.

Therefore:

```
map.get("A")
```

returning:

```
null
```

could mean:

```
A is absent
```

or:

```
A exists with null value
```

Use:

```
map.containsKey("A")
```

to distinguish.

---

65. Null Key Internals

HashMap allows one null key.

```
map.put(null, "value");
```

works.

Internally, the null key is handled specially rather than calling:

```
null.hashCode()
```

because that would cause a NullPointerException.

---

66. Null Value

HashMap also allows:

```
map.put("A", null);
```

So:

```
null key   → one allowed
null value → allowed
```

This differs from ConcurrentHashMap.

---

67. Mutable Key Problem

This is one of the most important HashMap interview topics.

Suppose:

```
class Employee {
    int id;

    @Override
    public int hashCode() {
        return id;
    }

    @Override
    public boolean equals(Object obj) {
        ...
    }
}
```

Then:

```
Employee e = new Employee(10);

map.put(e, "John");
```

Now:

```
e.id = 20;
```

What happens?

The object remains physically in its old bucket, but its logical hash may now point to a different bucket.

---

68. Mutable Key Problem Visualization

Initially:

```
Employee id = 10
       ↓
hash = 10
       ↓
bucket 10
```

After mutation:

```
Employee id = 20
       ↓
hash = 20
       ↓
bucket 20
```

But the existing entry is still sitting in:

```
bucket 10
```

So:

```
map.get(e)
```

may fail.

---

69. Why Does `get()` Fail?

Because HashMap calculates the current hash of `e`:

```
20
```

and searches:

```
bucket 20
```

But the entry was originally stored in:

```
bucket 10
```

Therefore it isn't found.

---

70. Best Practice

Use immutable keys.

Good examples:

```
String
Integer
Long
Enum
records with immutable components
```

or carefully designed immutable domain objects.

---

71. `equals()` Must Be Consistent

Another issue:

If key equality changes after insertion, Map behavior can become unpredictable.

Therefore keys should ideally be:

```
immutable
+
stable equals()
+
stable hashCode()
```

while stored as keys.

---

72. Why String Is an Excellent HashMap Key

`String` is effectively immutable.

Its:

```
equals()
hashCode()
```

behavior remains stable after insertion.

That's one reason Strings are commonly used as keys.

---

73. Java 7 HashMap Infinite Loop Issue

A historical interview topic:

> Why could Java 7 HashMap experience an infinite loop during concurrent resize?

The old implementation could transfer linked-list nodes in a way that allowed concurrent resize operations to create a cyclic linked structure.

Conceptually:

```
A → B → C
```

could become:

```
A → B → C
    ↑   ↓
    └───┘
```

Then traversal could loop forever.

---

74. Java 8 Improvement

Java 8 changed the resize mechanics and introduced tree bins.

The historical infinite-loop problem associated with concurrent HashMap resizing was addressed by the changed implementation.

But:

> **HashMap is still not thread-safe.**

Do not conclude that Java 8 made HashMap safe for concurrent modification.

---

75. HashMap Concurrent Modification

Even though modern HashMap is improved internally:

```
HashMap
```

still should not be concurrently mutated by multiple threads without external synchronization.

Use:

```
ConcurrentHashMap
```

for concurrent Map requirements.

---

76. Fail-Fast Iterator

HashMap iterators are generally:

> **Fail-fast**

Example:

```
for (String key : map.keySet()) {
    map.put("X", 100);
}
```

This can cause:

```
ConcurrentModificationException
```

---

77. What Does Fail-Fast Mean?

The iterator detects certain structural modifications made outside the iterator and may throw:

```
ConcurrentModificationException
```

The purpose is to detect programming errors early.

It is **not** a thread-safety mechanism.

---

78. `modCount`

HashMap internally maintains a modification count concept.

Iterators can compare their expected modification state against the Map's current modification state.

Conceptually:

```
iterator expectedModCount
          vs
map modCount
```

If they differ unexpectedly:

```
ConcurrentModificationException
```

may be thrown.

---

79. Fail-Fast Is Best-Effort

Very important:

> Fail-fast behavior is not guaranteed as a synchronization mechanism.

The JDK documentation describes it as best-effort.

You should never write business logic that depends on:

```
ConcurrentModificationException
```

being thrown.

---

80. Iterator Removal

If you're iterating:

```
Iterator<String> iterator =
    map.keySet().iterator();
```

and want to remove the current entry, use:

```
iterator.remove();
```

rather than directly modifying the Map.

This is the supported iterator-removal mechanism.

---

81. Structural vs Non-Structural Modification

This is useful for interviews.

A structural modification changes the Map's structure, such as:

```
adding a mapping
removing a mapping
```

Simply replacing the value of an existing mapping isn't necessarily considered a structural modification.

Therefore:

```
map.put(existingKey, newValue);
```

may not have the same iterator implications as adding/removing an entry.

---

82. HashMap Treeification and Comparable

Tree bins need a way to order nodes.

When keys are comparable in a suitable way, their ordering can help the tree structure.

If keys aren't naturally comparable, HashMap has additional tie-breaking mechanisms.

The key interview takeaway:

> Treeification doesn't require your key class to implement `Comparable`.

HashMap can still maintain its tree structure using internal tie-breaking logic.

---

83. Do Keys Need `Comparable`?

No.

This is a common misconception.

You can use:

```
HashMap<MyClass, String>
```

without:

```
MyClass implements Comparable<MyClass>
```

Treeification can still occur.

---

84. Why Doesn't HashMap Require Comparable?

Because HashMap isn't a sorted collection.

The tree is an internal collision-management mechanism, not a user-visible sorting feature.

---

85. HashMap Tree Bin ≠ Sorted Map

This is extremely important.

If you need:

```
sorted keys
```

use:

```
TreeMap
```

Tree bins exist only to manage collisions efficiently.

---

86. Worst-Case Complexity

Before treeification:

```
heavily collided bin
→ O(n)
```

With a tree bin:

```
tree lookup
→ approximately O(log n)
```

Overall HashMap remains approximately:

```
average → O(1)
```

for normal operations with good hashing.

---

87. Does Treeification Make HashMap O(log n)?

No.

This is another interview trap.

HashMap is still:

```
average O(1)
```

The tree only affects individual heavily-collided bins.

You should not describe HashMap as:

```
O(log n)
```

just because tree bins exist.

---

88. Why HashMap Needs Both Hash and Equals

Imagine:

```
100,000 entries
```

If HashMap used only:

```
equals()
```

it might have to compare the requested key against every entry.

Hashing narrows the search:

```
100,000 entries
      ↓
one bucket
      ↓
few candidates
```

Then `equals()` identifies the exact key.

Therefore:

```
hashCode()
→ narrow search space

equals()
→ confirm exact key
```

---

89. HashMap Performance Depends on Good `hashCode()`

Bad:

```
@Override
public int hashCode() {
    return 1;
}
```

Every key maps to the same bucket.

Conceptually:

```
bucket 1
 ↓
A → B → C → D → E → ...
```

This creates heavy collision behavior.

Modern HashMap can treeify such bins when conditions permit, but a good hash function is still essential.

---

90. Constant Hash Code Example

Suppose:

```
class BadKey {
    int id;

    @Override
    public int hashCode() {
        return 1;
    }

    @Override
    public boolean equals(Object obj) {
        return this == obj;
    }
}
```

Every key goes to the same bucket.

This is terrible for distribution.

---

91. HashMap Doesn't Guarantee Order

Don't depend on:

```
insertion order
```

or:

```
sorted order
```

Example:

```
map.put("A", 1);
map.put("B", 2);
map.put("C", 3);
```

The iteration order is not guaranteed to be:

```
A
B
C
```

Use:

```
LinkedHashMap
```

for insertion/access order.

Use:

```
TreeMap
```

for sorted order.

---

92. HashMap Is Not Synchronized

This is fundamental:

```
HashMap
→ not thread-safe
```

If multiple threads modify it concurrently, use:

```
ConcurrentHashMap
```

or another appropriate synchronization strategy.

---

93. `Collections.synchronizedMap()`

You can wrap:

```
Map<K,V> map =
    Collections.synchronizedMap(
```

```
        new HashMap<>()
    );
```

This provides synchronized access around Map operations.

But iteration requires proper external synchronization according to the wrapper's contract.

For high-concurrency applications:

```
ConcurrentHashMap
```

is usually the better fit.

---

94. HashMap Memory Model

Conceptually:

```
HashMap
 |
 ↓
Node[] table
 |
 ├── bucket 0
 ├── bucket 1 → Node
 ├── bucket 2
 ├── bucket 3 → Node → Node
 └── ...
```

As entries grow:

```
size
 ↓
threshold
 ↓
resize
 ↓
larger table
```

---

95. Complete `put()` Interview Answer

If an interviewer asks:

> "Explain HashMap put() internally."

Answer:

> HashMap first computes the key's hash code and applies hash spreading. It then calculates the bucket index using `(n - 1) & hash`, where the table capacity is a power of two. If the bucket is empty, a node is inserted. If it is occupied, HashMap checks the hash and key equality to determine whether the key already exists; if so, its value is replaced. Otherwise the new entry is added to the collision structure. If the bin becomes sufficiently large, it may be treeified, provided the table is large enough. After insertion, if the size exceeds the resize threshold, the table is resized and entries are redistributed.

That is a **senior-level answer**.

---

96. Complete `get()` Interview Answer

> HashMap computes and spreads the key's hash, determines the bucket index, then searches the corresponding bin. It compares candidate hashes and uses key equality to identify the exact mapping. If the key is found, its value is returned; otherwise null is returned.

---

97. Complete Collision Answer

> A collision occurs when different keys map to the same bucket. HashMap stores those entries in the same bin, historically as a linked list and, in modern Java, potentially as a Red-Black Tree when the collision count and table-capacity conditions justify treeification.

---

98. Complete Rehashing Answer

> When the number of mappings crosses the resize threshold, HashMap increases the table capacity, normally doubling it. Because the capacity remains a power of two, entries can be efficiently redistributed between their old index and old index plus the old capacity based on the relevant hash bit. The key's original hashCode doesn't need to be conceptually recomputed from scratch for every entry.

---

99. Complete Load Factor Answer

> Load factor controls how full the hash table is allowed to become before resizing. The default is 0.75. A lower load factor generally reduces collisions at the cost of memory, while a higher load factor saves memory but can increase collisions.

---

100. Complete Treeification Answer

> When a bin becomes sufficiently collision-heavy, around eight nodes, HashMap may convert that bin into a Red-Black Tree. However, it first considers table capacity; if the table is too small, resizing is preferred. The commonly cited minimum treeification capacity is 64.

---

101. ⭐ Interview Question

Why does HashMap use a power-of-two capacity?

Answer:

> So bucket indexing can efficiently use `(n - 1) & hash`, and doubling the capacity during resize allows entries to either remain at the same index or move by the old capacity based on one additional hash bit.

---

102. ⭐ Interview Question

What is the difference between capacity, size and threshold?

```
Capacity
→ number of buckets

Size
→ number of mappings

Threshold
→ approximate size at which resize occurs
```

And:

```
threshold ≈ capacity × load factor
```

---

103. ⭐ Interview Question

What is the default load factor?

```
0.75
```

---

104. ⭐ Interview Question

What is the default initial capacity?

Common default:

```
16
```

with lazy table initialization.

---

105. ⭐ Interview Question

Why isn't a collision a problem?

Collisions are expected in hash tables.

HashMap handles them through:

```
linked nodes
```

and, for sufficiently collision-heavy bins under the right conditions:

```
Red-Black Trees
```

The goal is to keep average performance efficient.

---

106. ⭐ Interview Question

What happens when two keys have the same hashCode?

They may map to the same bucket.

HashMap then uses key equality to determine whether they represent the same logical key.

If:

```
equals() == false
```

both entries can coexist.

---

107. ⭐ Interview Question

What happens if two keys have the same hashCode and `equals()` returns true?

They represent the same logical key.

The new value replaces the existing value.

Size does not increase.

---

108. ⭐ Interview Question

Can two unequal keys have the same hashCode?

Yes.

That's a collision.

```
hashCode(A) == hashCode(B)
A.equals(B) == false
```

is perfectly valid.

---

109. ⭐ Interview Question

If two objects have the same hashCode, are they equal?

No.

The correct implication is:

```
equals() == true
       ↓
hashCode() must be same
```

But:

```
hashCode() same
       ↔
equals() true
```

---

110. ⭐ Interview Question

Why should HashMap keys be immutable?

Because changing fields used by `equals()` or `hashCode()` after insertion can cause the Map to search a different bucket from the one where the entry was stored.

---

111. ⭐ Interview Question

Can HashMap have a null key?

Yes.

It supports one null key.

---

112. ⭐ Interview Question

Can HashMap have multiple null values?

Yes.

Example:

```
map.put("A", null);
map.put("B", null);
```

Both are valid.

---

113. ⭐ Interview Question

Is HashMap thread-safe?

No.

Concurrent mutation requires external synchronization or a concurrent collection such as:

```
ConcurrentHashMap
```

---

114. ⭐ Interview Question

Why doesn't HashMap use `hashCode()` directly as the index?

Because hashCode is a general integer and isn't necessarily in the range:

```
0 ... capacity - 1
```

HashMap spreads the hash and maps it to a valid bucket index.

---

115. ⭐ Interview Question

Why does HashMap use `(n - 1) & hash`?

Because its table size is maintained as a power of two, allowing efficient bit masking to calculate the bucket index.

---

116. ⭐ Interview Question

What is treeification?

Converting a collision-heavy bucket from a linked-node representation into a tree-based representation.

---

117. ⭐ Interview Question

Is every bucket a tree?

No.

Only sufficiently collision-heavy bins can become trees.

Most bins remain:

```
empty
```

or:

```
single node
```

or:

```
linked nodes
```

---

118. ⭐ Interview Question

Is HashMap internally a Red-Black Tree?

No.

It is primarily a hash table:

```
Node[]
```

with individual collision-heavy bins potentially represented as Red-Black Trees.

---

119. ⭐ Interview Question

Does treeification make HashMap sorted?

No.

Tree bins are internal collision-management structures.

HashMap still doesn't guarantee sorted iteration.

---

120. ⭐ Interview Question

What is `TREEIFY_THRESHOLD`?

Common JDK implementation constant:

```
8
```

It represents the bin size at which treeification may be considered.

But:

```
MIN_TREEIFY_CAPACITY = 64
```

also matters.

---

121. ⭐ Interview Question

What is `UNTREEIFY_THRESHOLD`?

Common implementation value:

```
6
```

It is associated with converting a tree bin back to a linked representation when the bin becomes sufficiently small.

---

122. ⭐ Interview Question

Why does HashMap resize before treeifying in some cases?

If the table is too small, collisions may be caused primarily by insufficient capacity.

Increasing capacity can distribute entries across more buckets more cheaply than immediately creating a tree.

---

123. ⭐ Interview Question

Does HashMap recompute hashCode during resize?

Don't say:

> "It calls `hashCode()` again for every key."

A better answer:

> The existing hash values are reused and entries are redistributed according to the new capacity. Java 8+ uses an efficient high-bit split during doubling.

---

124. ⭐ Interview Question

Why does resizing normally double capacity?

It preserves the power-of-two property and enables efficient redistribution using the old index and the old capacity offset.

125. ⭐ Interview Question

What is the worst-case lookup complexity?

For a heavily collided linked bin:

```
O(n)
```

For a treeified bin:

```
O(log n)
```

Average HashMap lookup with good hashing:

```
O(1)
```

---

126. ⭐ Interview Question

What is the difference between HashMap and ConcurrentHashMap?

```
HashMap
→ not thread-safe
→ permits null key/value
→ fail-fast iterator

ConcurrentHashMap
→ thread-safe
→ no null key/value
→ weakly consistent iterator
→ designed for concurrent access
```

---

127. ⭐ Interview Question

What happens if HashMap is modified while iterating?

A fail-fast iterator may throw:

```
ConcurrentModificationException
```

for certain structural modifications.

But this behavior is best-effort, not a concurrency guarantee.

---

128. ⭐ Coding Question

What is the output?

```
Map<String, Integer> map =
    new HashMap<>();

map.put("A", 10);
map.put("A", 20);

System.out.println(map.size());
System.out.println(map.get("A"));
```

Answer:

```
1
20
```

---

129. ⭐ Coding Question

```
Map<String, Integer> map =
    new HashMap<>();

map.put("A", null);

System.out.println(
    map.get("A")
);

System.out.println(
```

```
    map.containsKey("A")
);
```

Output:

```
null
true
```

---

130. ⭐ Coding Question

```
Map<String, Integer> map =
    new HashMap<>();

System.out.println(
    map.get("A")
);

System.out.println(
    map.containsKey("A")
);
```

Output:

```
null
false
```

This demonstrates the null-value ambiguity.

---

131. ⭐ Coding Question

Suppose:

```
A.hashCode() = 100
B.hashCode() = 100
A.equals(B) = false
```

Can both exist in HashMap?

Yes.

They are different logical keys despite having the same hash.

---

132. ⭐ Coding Question

Suppose:

```
A.hashCode() = 100
B.hashCode() = 200
A.equals(B) = true
```

Is this a valid key implementation?

No.

It violates:

```
equal objects → same hashCode
```

---

133. ⭐ Coding Question — Mutable Key

```
Employee e =
    new Employee(10);

map.put(e, "John");

e.setId(20);

System.out.println(
    map.get(e)
);
```

Can this return:

```
null
```

Yes, if `id` participates in `hashCode()` and `equals()`.

This is the classic mutable-key problem.

---

134. ⭐ Coding Question — Collision

Suppose all keys return:

```
@Override
public int hashCode() {
    return 1;
}
```

What happens?

All keys tend toward the same bucket.

Modern HashMap can treeify the heavily collided bin when its capacity and threshold conditions permit.

But performance is still much worse than having a well-distributed hash function.

---

135. ⭐ Coding Question — Treeification

Suppose a bin reaches 8 entries.

Does it automatically become a tree?

Not necessarily.

If the table capacity is below the required minimum:

```
64
```

HashMap may resize instead.

---

136. ⭐ Coding Question — Capacity

Suppose:

```
capacity = 16
load factor = 0.75
```

Threshold is approximately:

```
16 × 0.75 = 12
```

When the size exceeds the threshold, resize is triggered.

---

137. ⭐ Coding Question — Resize

Suppose:

```
old capacity = 16
```

After resizing:

```
new capacity = 32
```

An entry from bucket:

```
5
```

can generally remain at:

```
5
```

or move to:

```
21
```

because:

```
5 + 16 = 21
```

---

138. ⭐ Senior-Level Question

Why does HashMap use hash spreading if hashCode is already provided?

Answer:

> HashCode implementations may not distribute useful information evenly in the low bits. Because bucket selection with power-of-two capacity depends heavily on low bits, HashMap mixes higher bits into lower bits to improve distribution.

---

139. ⭐ Senior-Level Question

Why is `equals()` called after locating a bucket?

Because multiple keys can map to the same bucket.

Hashing narrows the search:

```
all entries
 ↓
one bucket
```

Then equality determines:

```
which key exactly?
```

---

140. ⭐ Senior-Level Question

Why can equal objects not be placed in different buckets?

They shouldn't be, assuming the `equals()` / `hashCode()` contract is correctly implemented.

If:

```
a.equals(b)
```

then:

```
a.hashCode() == b.hashCode()
```

and they should follow the same hash/bucket calculation.

---

141. ⭐ Senior-Level Question

Why can unequal objects be in the same bucket?

Because a finite number of buckets must accommodate potentially many possible hash values.

Therefore collisions are inevitable.

---

142. ⭐ Senior-Level Question

Why doesn't HashMap simply use a LinkedList for everything?

Because a long collision chain can degrade lookup toward:

```
O(n)
```

Treeification provides better worst-case behavior for heavily collided bins.

---

143. ⭐ Senior-Level Question

Why doesn't HashMap use a tree for the entire Map?

Because hashing gives excellent average O(1) lookup.

A global tree would unnecessarily turn normal lookups into:

```
O(log n)
```

The tree is only needed for problematic individual bins.

---

144. ⭐ Senior-Level Question

Why doesn't HashMap automatically resize after every collision?

Because resizing costs time and memory.

A few collisions are normal.

HashMap balances:

```
table size
+
collision rate
+
memory
```

using capacity and load factor.

---

145. ⭐ Senior-Level Question

What happens if load factor is 1.0?

The Map can tolerate a fuller table before resizing.

This may reduce memory usage but can increase collisions.

It isn't automatically better or worse; it's a trade-off.

---

146. ⭐ Senior-Level Question

Why is 0.75 commonly used?

It provides a practical balance between:

```
space efficiency
```

and:

```
lookup performance
```

---

147. ⭐ Senior-Level Question

What is the difference between resizing and treeification?

```
Resizing
→ increases number of buckets

Treeification
→ changes one heavily-collided bin into a tree
```

They solve different problems.

---

148. ⭐ Senior-Level Question

Can a tree bin become a linked list again?

Yes.

When the number of nodes falls sufficiently, the bin can be untreeified.

---

149. ⭐ Senior-Level Question

Does HashMap use synchronization internally?

Normal HashMap is not designed as a thread-safe Map.

Don't rely on its internal mechanics for synchronization.

Use:

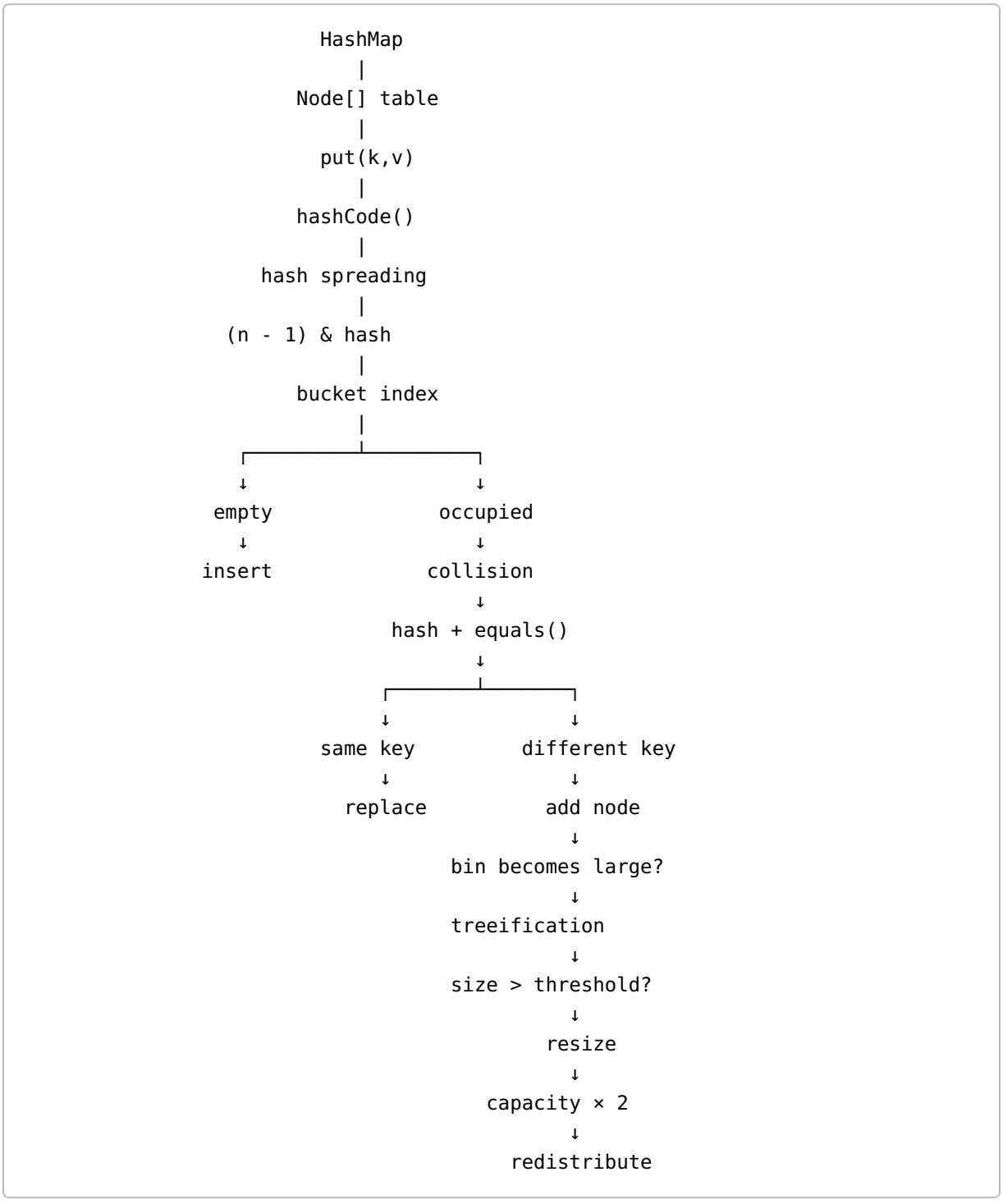
```
ConcurrentHashMap
```

when concurrent access is required.

---

150. 🔥 The Complete HashMap Mental Model

Memorize this:



If you can explain this diagram naturally, you understand HashMap internals at an interview-ready level.

🏆 HashMap Interview Cheat Sheet

Concept	Remember
Structure	Hash table
Main array	`Node[] table`
Hash	`hashCode()` + spreading
Spread	`h ^ (h >>> 16)` concept
Bucket index	`(n - 1) & hash`
Capacity	Power of two
Default initial capacity	16
Default load factor	0.75
Threshold	capacity × load factor
Collision	Different keys → same bucket
Duplicate key	`equals()` true → replace value
Collision structure	Linked nodes / tree
Tree type	Red-Black Tree
Tree threshold	8
Minimum treeify capacity	64
Untreeify threshold	Commonly 6
Average lookup	O(1)
Heavy linked collision	O(n)
Tree bin lookup	O(log n) approximately
Resize	Usually doubles capacity
Resize redistribution	Old index or old index + old capacity
Null key	One allowed
Null value	Allowed
Ordering	None guaranteed
Thread-safe	❌
Iterator	Fail-fast, best-effort

Concept	Remember
Mutable keys	Dangerous
`Comparable` required for treeification?	❌
Tree bins make HashMap sorted?	❌

🎯 Final Interview Formula

When asked:

> **"How does HashMap work?"**

Don't give a 30-second shallow answer.

Use this sequence:

```
1. HashMap maintains a bucket array.

2. On put/get, it obtains the key's hashCode.

3. The hash is spread to improve distribution.

4. Bucket index is calculated using:
   (n - 1) & hash

5. If bucket is empty, insert.

6. If occupied, compare hashes and keys
   using equals().

7. Same key → replace value.

8. Different key → collision structure.

9. Heavy collision → treeification
   when conditions are met.

10. When size exceeds threshold:
    resize table.

11. Capacity normally doubles.

12. Entries are redistributed efficiently
```

```
    using the additional hash bit.

13. Average lookup is O(1).

14. HashMap is not thread-safe.
```

That answer demonstrates both **API-level understanding and implementation-level knowledge**.

🚀 Next Chapter

Chapter 23 — HashMap Interview Problems & Advanced Traps

We'll move from **"How HashMap works"** to **"Can you actually solve HashMap interview problems?"**

Topics:

- `HashMap` output-based questions
- `equals()` / `hashCode()` traps
- Mutable-key puzzles
- Custom key interview problems
- Collision puzzles
- `put()` replacement scenarios
- `get()` returning null traps
- HashMap resizing questions
- Capacity calculation problems
- Load-factor questions
- Treeification scenarios
- Java 7 vs Java 8+ questions
- Fail-fast iterator traps
- Concurrent modification scenarios
- `HashMap` memory questions
- `HashMap` vs `ConcurrentHashMap`
- `HashMap` vs `LinkedHashMap`
- `HashMap` vs `TreeMap`
- LRU cache using `LinkedHashMap`
- Frequency-map problems
- Two-sum patterns
- Grouping/anagram problems
- Top-K patterns
- Senior-level HashMap design questions
- **30+ interview coding problems with solutions**